Evaluating GPT-5.4-mini for Gherkin Test Generation: An Empirical Study

Nguyen Minh Hoang, Nguyen Hieu An, and Le Thi Quynh Chi*

FPT University, Vietnam
chiltq6@fe.edu.vn

Abstract. Large language models can draft behavior-driven development (BDD) scenarios, but valid syntax does not guarantee preserved business intent. We evaluate GPT-5.4-mini on reconstructing 261 project-maintained Gherkin files from Sylius, Apache Fineract and Diaspora from user stories reverse-engineered from them, with one same-project example per prompt. We measure parser compliance with behave and skeleton cosine similarity to the reference. Excluding the three prompt exemplars ($N = 258$), 255 files (98.8%, 95% CI [96.6%, 99.8%]) are valid Gherkin, above the 80% threshold ($p < 0.001$). The median similarity is 0.853; a one-sided exact sign test does not show that it exceeds the proposal's 0.85 reference point ($p = 0.209$, $\delta = 0.054$). Only the e-commerce project exceeds it; the decision changes at 0.84. In an exploratory audit of 60 high-similarity files, 25 contain a business-logic defect (AI-drafted, human-reviewed labels; $\kappa = 0.571$ and 0.609 against an independent rater), and seeded defects barely change the score. In a same-time rerun, removing the example lowers the median to 0.811, while another exemplar moves single files in both directions with no detectable average effect. Within this reconstruction setting, GPT-5.4-mini is a drafting aid whose business logic needs human review, and embedding similarity alone cannot certify generated Gherkin.

Keywords: Large language models · Gherkin · Behavior-driven development · Acceptance testing · Software testing · GPT-5.4-mini

1 Introduction

Behavior-driven development expresses expected software behavior in executable or executable-adjacent scenarios, commonly using Gherkin. Its cost is not limited to writing Given–When–Then syntax: engineers must preserve domain preconditions, project conventions, data dependencies and expected outcomes while keeping feature files parsable by the project toolchain. In the three projects of this study, a Sylius checkout scenario may depend on channel, country, shipping-method state and confirmation behavior, Apache Fineract scenarios encode dates, repayment periods, balances and data tables, and Diaspora scenarios describe browser-visible behavior such as navigation, tags, comments and notifications.

* Corresponding author.

A generated scenario can therefore be syntactically valid yet incomplete as a specification of business behavior, and converting a compact Connextra story into project-consistent Gherkin is a constrained generation problem rather than a simple paraphrasing task.

1.1 Research Gap and Motivation

Prior work shows that LLMs can generate BDD artifacts [1,2,3,4], but the evaluations differ substantially in data scale, model family and validation method (Section 2). These studies establish feasibility, but they do not jointly answer whether a current compact model can produce both parser-ready and structurally close Gherkin across multiple public software domains. In particular, none of the five studies in Table S5 of the supplementary material tests whether its semantic measure reacts to business-logic defects, and none varies the example given in the prompt.

This study addresses that narrower gap by evaluating the pinned gpt-5.4-mini-2026-03-17 snapshot on 261 stories from three unrelated open-source projects, using same-project one-shot prompting and two separate outcome measures. The contribution is not a claim that GPT-5.4-mini is superior to earlier models, but an auditable evaluation design that separates syntactic validity from structural similarity on a public, multi-domain benchmark and tests how far the similarity metric can be trusted. Because the stories were reverse-engineered from the reference files and the 0.85 reference point was not calibrated, this study therefore includes a leakage measurement, a defect audit, a perturbation test, a reference-point sensitivity analysis, per-project results, a prompt ablation and a same-time rerun.

1.2 Research Questions and Contributions

RQ1 – Structural skeleton resemblance. When GPT-5.4-mini reconstructs feature files from user stories reverse-engineered from them, how is the skeleton cosine similarity to the reference distributed around the reference point of 0.85 fixed in the proposal?
RQ2 – Syntactic validity. Does GPT-5.4-mini generate feature files that the behave Gherkin parser accepts at a rate above 80%?

The paper makes three contributions: (1) a 261-case, multi-domain benchmark derived from project-maintained reference files in Sylius, Apache Fineract, and Diaspora, with its construction and leakage risk documented; (2) an evaluation that reports parser compliance separately from structural similarity and checks the similarity metric against defect labels, seeded defects and alternative reference points; and (3) a replication package containing the pinned model identifier, prompts, scripts, API logs, outputs, and all additional analyses.

Section 2 reviews related work. Section 3 describes the dataset, the generation pipeline, the metrics and the analyses, and Section 4 reports the results. Sections 5 and 6 discuss the implications and the threats to validity, and Section 7 concludes. The supplementary material, linked under Data and Ethics, gives the prompts

and additional results (Section S1), the perturbation test (Section S2) and further method and result details (Section S3).

2 Related Work

2.1 LLM-Based Gherkin and BDD Generation

Research on LLM-assisted BDD generation has typically emphasized either semantic resemblance or syntactic correctness; Table S5 of the supplementary material summarizes the five studies closest to ours. Rathnayake et al. [1] evaluated GPT-4, Claude 3 and Gemini on 500 user stories from one company and reported a BERTScore F1 of 91.16% for GPT-4. Fernandes et al. [4] found business-logic flaws, such as a reversed or omitted outcome, among the most frequent errors of seven LLMs. Karpurapu et al. [2] found that zero-shot outputs accounted for 89% of the observed Gherkin-lint syntax errors, but their validator was a linter rather than a runtime parser. In industrial workflows, all 50 Cypress test cases of Ferreira et al. [3] were valid TypeScript, yet only 60% reflected their Gherkin scenarios, and Fonseca et al. [5] found 93.3% of their generated scenarios syntactically valid by a manual keyword check. These single-organization or single-application evaluations show that LLMs can be integrated into practical workflows, but they make cross-domain generalization difficult to assess.

2.2 Test Generation and Semantic Evaluation

Broader software-test generation research shows that syntactic success and fault-revealing or semantic quality should not be conflated. Benchmarks exist for class-level test-case generation [6] and for test-case generation over generated program structures [7], and XUAT-Copilot uses LLM-based agents for user-acceptance testing [8]. For generated code, EvalPlus found pass@k up to 19.3–28.9% lower once the HumanEval tests were extended 80-fold [9], and CodeJudge shows that token- and embedding-based metrics fail on semantically equivalent code written differently [10]. Where an executable oracle exists it replaces resemblance [11,12,13]; Gherkin generation has no such oracle, because a valid scenario can still state the wrong business rule.

For comparison with a reference, sentence embeddings such as Sentence-BERT [16] provide a scalable alternative to manual labeling, but concrete names and values in raw Gherkin can dominate textual resemblance without representing domain intent. The present study therefore masks quoted values and numbers and keeps only keyword lines before embedding, treats the resulting skeleton similarity as a proxy rather than a measure of business correctness, and tests how well that proxy tracks business-logic defects (Section 4.4).

Reporting is a further concern. A review of 74 LLM studies in requirements engineering found that only 16% used public datasets and 39% did not state how prompts were selected [14]. Guidelines for empirical software-engineering studies with LLMs require reporting the model version, configuration and date of use, publishing all prompts and explaining how few-shot examples were selected, and they name sensitivity to in-context examples as a validity threat [15].

2.3 Positioning of This Study

Table S5 compares this study with the five BDD-generation studies above on the criteria that bear on our contributions.

Contribution 1, the benchmark. The other studies with named sources each draw on one company, system or application, and two do not release their data (Table S5). Our 261 reference files are from three public projects in different domains and are released with their construction and a leakage measurement.

Contribution 2, the evaluation. Karpurapu et al. check syntax with a linter, and the other studies measure semantics with similarity metrics, ratings or an LLM validator. Fernandes et al. describe business-logic flaws qualitatively, but none of the five studies tests whether its semantic measure reacts to business-logic defects, and none varies which example is given in the prompt. Our defect audit and perturbation test show that files with high skeleton similarity can still violate their acceptance criteria, and our ablation shows that removing the example lowers skeleton similarity by more than repeated runs vary, while a different exemplar moves single files in both directions without a detectable average effect.

Contribution 3, the replication package. It contains the model identifier, prompts, scripts, API logs, generated outputs and all additional analyses, so that every number in this paper can be recomputed.

3 Methodology

Fig. S3 in the supplementary material shows how the files pass through generation and analysis, with the denominator of each analysis.

3.1 Dataset Construction

The dataset pairs 261 project-maintained reference files (.feature files) from three open-source projects with 261 user stories. The stories were not collected from a backlog. They were reverse-engineered from the feature files: GPT-5.4-mini (temperature 0) received the text of each feature file and was asked to write one Connextra-style story ("As a [role], I want [feature], so that [benefit]") with as many acceptance criteria as needed to cover all of its scenarios. Only the first 15,000 characters of each file were passed to the model. This truncated 84 of the 261 files, all of them from Apache Fineract (84 of its 100 files, whose stored references have a median length of about 30,000 characters). Each generated file is scored against the whole stored reference, not against that prefix; for 23 Fineract files the stored reference is itself shorter than the upstream file. Sections 6 and 4.5 discuss and measure the leakage risk this creates.

Sources and selection. The files were collected in two batches: a random sample of 100 files downloaded on 28 June 2026 and 161 further files drawn at random from the clones of Table 1. In the first batch some stored references differ from their upstream files; scoring against the full upstream files leaves every RQ1 decision unchanged.

Table 1. Dataset composition and sources. The commits are the heads of the clones made on 19 July 2026; .feature counts all feature files at that commit. The three exemplars of the original prompt are excluded from the primary analysis.

Project	Domain	Branch, commit	.feature	Selected	Analysed
Sylius [17]	E-commerce	2.3, `9ab28ca`	839	100	99
Fineract [18]	Finance	develop, `5a93304`	147	100	99
Diaspora [19]	Social network	develop, `b7bf1dc`	71	61	60
Total			1,057	261	258

The projects were chosen for domain and style diversity (Section 1). The benchmark is a deliberately heterogeneous sample of mature public English-language repositories, not a representative sample of BDD practice.

3.2 Model and Prompt Configuration

All generation requests specified gpt-5.4-mini-2026-03-17 [20] through the OpenAI Chat Completions API [21] with temperature 0; the returned model name was recorded for C1, C2 and the rerun below but not for the original C0. The requests set no maximum-token cap, which deviates from the research proposal's planned cap of 1,500 output tokens. The original condition (C0) is same-project one-shot prompting: each project has its own system message and one example pair (user story and reference Gherkin) from the same project (Listing S1 in the supplementary material). The three example files (ids 2, 102 and 202) are also members of the dataset, and their own generated files score 1.000, 0.997 and 1.000 on the metric below. The primary analysis therefore excludes them ($N = 258$); results for all 261 files are reported alongside, and no generated file was excluded because of its content.

Prompt ablation. C1 is zero-shot: the same system message and wording, with the example block and the sentence referring to it removed. C2 is one-shot with a different exemplar per project (ids 86, 119 and 203), drawn with a fixed random seed (2026) from the remaining files of each project before any C2 output existed. Model, temperature, system messages and output clean-up are identical in all conditions, and the ablation is analysed on the 255 files that are not an exemplar in any condition. Its analysis plan, written before the C1 and C2 runs of 16 October 2026 and released with its SHA-256 hash, specifies paired Wilcoxon signed-rank tests on per-file cosine and exact McNemar tests on parser outcomes, each against C0 and Holm-corrected. All three conditions were scored with an ONNX export of the encoder that reproduces the original C0 scores.

Same-time rerun. Because the original C0 ran on 19 July 2026, three months before C1 and C2, we ran C0, C1 and C2 again on 28 October 2026 with one script and one logging path, on all 261 stories and four more times on a subset of 60 stories (20 per project, drawn with a fixed seed from the files that are not an exemplar in C0 or C2). The same run included C0cap, C0 with the proposal's cap of 1,500 output tokens, on all 261 stories (Section 6). The 1,764 requests were shuffled with a fixed seed and sent by four parallel workers, so

the conditions are interleaved in time. The analysis plan, committed with its SHA-256 hash before the first request, repeats the ablation tests against the new C0 and measures run-to-run variation on the 60 files. It fixes that the original C0 stays the primary result and that a paired median difference smaller than the pooled within-file SD is not reported as a prompt effect.

3.3 Generation and Recording Pipeline

The pipeline builds the prompt for the story's project, calls the API and removes a Markdown code fence only when the model wraps its output in one; no generated text is edited by hand, and each call is logged with its timestamp and token counts. The C0 notebook used a timeout of 600 s and retried any failed request, including rate-limit errors, after a 60 s wait with no limit on the number of attempts; the C1, C2 and rerun scripts made up to five attempts per story with waits between 5 and 60 s. These scripts treated an empty output as a failure, and a story still failing after a resumed run would have stayed in the denominator as a parser FAIL. None of these events occurred: all C1, C2 and rerun requests succeeded at the first attempt, all 261 C0 requests returned an output, and no output in any condition was empty.

3.4 Metric 1: Skeleton Cosine Similarity

Skeleton cosine similarity compares a generated file with its reference after reducing both to a skeleton. The skeleton keeps every line that begins with a Gherkin keyword (Feature, Background, Scenario, Given, When, Then, And, But), including the keyword itself. It removes tags, replaces every double-quoted string with `<PARAM>` and every number with `<NUM>`, and drops all other lines, so data tables, doc strings and Examples tables disappear (Table S2 in the supplementary material). Each skeleton is encoded as a whole with all-MiniLM-L6-v2 [16,22], which mean-pools token embeddings and truncates its input at 256 word-piece tokens. With g and r the embeddings of the generated and reference skeletons,

$$\mathrm{SkelCos}(g, r) = \frac{g \cdot r}{\|g\|\|r\|}. \tag{1}$$

Two properties of this design limit what the metric can show. First, masking makes the score blind to changed numbers and quoted values by construction. Second, truncation means that the score often reflects only the start of a file: 174 of the 258 reference skeletons (67.4%) and 166 of the 258 generated skeletons (64.3%) exceed 256 tokens, including 94 of the 99 Fineract references (median 3,506 tokens). The metric therefore measures the structural resemblance of keyword lines to the reference, not the preservation of business intent; evidence on business intent comes only from the defect audit (Section 3.6), against which Section 4.4 checks the metric.

The reference point of 0.85 was fixed in the research proposal before the full run. It was not calibrated against human judgement, and it is not an acceptance

criterion: no labels independent of the scores were available to calibrate a cut-off. No prior study reports skeleton cosine with this encoder for Gherkin, and the proposal's pilot (30 stories, ids 1–10, 101–110 and 201–210, mean 0.859) used stories that are also part of the evaluated set. We therefore treat 0.85 as an operational reference point and report how the conclusions change between 0.80 and 0.90 (Section 4.3).

3.5 Metric 2: Parser Compliance

Parser compliance is the share of generated files that the behave Gherkin parser accepts. For a generated set S,

$$\mathrm{ESR} = \frac{|\{s \in S \mid \mathrm{parse}(s) = \mathrm{PASS}\}|}{|S|}. \tag{2}$$

We keep the proposal's name, executable syntax rate (ESR), but the check is narrower: each file is passed to behave's parser (behave 1.3.3 [23]) without step definitions, so no step is matched or executed, and a PASS means only that the file is valid Gherkin, not that its scenarios are complete, executable against the application or correct. A dry run (behave --dry-run) is not used, because without a steps directory it stops with a configuration error before parsing the file. Versions 1.2.6 and 1.3.3 of behave agree on every generated file.

3.6 Defect Audit and Structural Checks

Because neither metric checks business intent, we audited high-scoring outputs for defects. Of the 258 analysed files, 136 have a skeleton cosine of at least 0.85. From these we drew 20 files per project with a fixed seed (2026), giving 60 files. Each file was labelled against its user story and acceptance criteria; the reference was consulted only for project conventions. For each criterion the labeller recorded a business-logic facet (numbers, conditions, outcomes) and a temporal/causal facet (preconditions, triggers, order of states). Each defect received a severity from 0 (none) to 3 (the test would pass on a system that violates the criterion), and a file counts as defective if it has at least one defect of severity 2 or higher. Line-level evidence was recorded for each defect.

The labels were drafted by an AI coding assistant (OpenAI Codex) and then reviewed by a second author, who retained all of them unchanged; that review was recorded as one confirmation covering all 60 files, not as a decision with reasons per file. To estimate reliability, the first author, who reports not having seen the labels or their aggregate rates before rating, labelled a subset of 15 files fixed in advance (6 Sylius, 3 Fineract, 6 Diaspora) with the same guide, and we report Cohen's κ [24] between these labels and the reviewed labels. This is agreement between one independent human rater and AI-drafted, human-reviewed labels, not between two independent human raters. Because both values fell below the 0.7 required by our protocol, the reviewing author and the independent rater then held the protocol's consensus round on the 15 files and recorded a final label, severity, reason and line evidence for each of the 30 file–facet decisions.

Separately, an automated check flags four keyword-order violations per scenario, and a perturbation test applied eight defect-seeding operators to 120 scenarios extracted from the reference files and compared each mutant with its original using the same metric (Section S2 of the supplementary material).

3.7 Statistical Analysis Plan

The confirmatory tests use $\alpha = 0.05$ and follow standard guidance on matching tests to data properties [25]. Effect sizes are reported without categorical labels such as small or large and are interpreted through their magnitude and 95% confidence intervals: an exact interval for δ, derived from the Clopper–Pearson interval of the share of files above 0.85, and percentile bootstrap intervals (10,000 resamples, seed 2026) for η^2_H, Spearman's ρ, Cohen's κ (resampling files) and, outside the rerun plan, the median paired differences of the ablation.

RQ1: exact sign test, with the Wilcoxon test as sensitivity analysis. H_0: the median skeleton cosine is at most 0.85; H_1: it exceeds 0.85. The proposal phrased these hypotheses in terms of the mean and pre-registered a one-sided one-sample Wilcoxon signed-rank test on $x_i - 0.85$, which addresses the median only if the differences are symmetric; they are not (Section 4). The median is therefore tested with the exact one-sided sign test [26] and estimated with a 95% percentile bootstrap interval [27], and the Wilcoxon test is reported as a sensitivity analysis. As effect size we report $\delta = P(x > 0.85) - P(x < 0.85)$. A non-significant result is read as insufficient evidence for performance above the reference point, not as evidence of poor performance.

RQ2: exact binomial test. Each file has a binary parser outcome. We test H_0: $p_{\mathrm{ESR}} \leq 0.80$ against H_1: $p_{\mathrm{ESR}} > 0.80$ [26] and report a 95% Clopper–Pearson interval [28] and the risk difference from 0.80.

Exploratory analyses. The following analyses were planned after the reviews and are exploratory. Per-project differences are tested with a Kruskal–Wallis test [29], reported with $\eta^2_H = (H - k + 1)/(n - k)$ and followed by pairwise two-sided Mann–Whitney tests with Holm correction [30]. Reference-point sensitivity repeats the RQ1 test for reference points from 0.80 to 0.90. Lexical leakage is measured as the share of distinct word trigrams of the reference that also occur in the story, related to skeleton cosine with Spearman's ρ.

4 Results

4.1 RQ1: Structural Skeleton Resemblance

On the 258 non-exemplar files the median is 0.853 (95% bootstrap CI 0.844–0.859), 0.003 above the reference point. 136 files lie above 0.85 and 122 below, and the exact sign test gives one-sided $p = 0.209$ with $\delta = 0.054$ (95% CI -0.072 to 0.179) (Table S6 and Fig. S1 in the supplementary material). The differences are skewed to the left (skewness -0.41, 95% bootstrap CI -0.76 to -0.08; the mean is 0.848), so the pre-registered Wilcoxon test addresses the

pseudo-median (Hodges–Lehmann estimate 0.850) rather than the median; it gives $p = 0.474$. Including the three exemplars gives $p = 0.161$ (sign test) and $p = 0.355$ (Wilcoxon). Decision: fail to reject H_0 in all four analyses.

4.2 Results per Project

The pooled median hides differences between projects (Table 2; Fig. S2 in the supplementary material). Sylius is the only project above the reference point: its median is 0.887 and 77.8% of its files score at least 0.85. Fineract (median 0.826) and Diaspora (median 0.831) are significantly below 0.85 by the Wilcoxon test (one-sided $p < 0.001$ and $p = 0.008$); with the sign test, Fineract remains significantly below ($p = 0.001$) but Diaspora does not ($p = 0.12$; 25 files above, 35 below). The projects differ (Kruskal–Wallis $H = 60.87$, df $= 2$, $p < 0.001$, $\eta^2_H = 0.231$, 95% CI 0.144–0.336). Sylius differs from Fineract and from Diaspora (Holm-adjusted $p < 0.001$ for both), while the difference between Fineract and Diaspora is not significant ($p = 0.74$).

Truncated story input. The 84 Fineract files whose stories saw only the first 15,000 characters of the reference do not score lower than the 15 whose stories saw all of it (median 0.827 vs. 0.824; two-sided Mann–Whitney $p = 0.99$), and rescoring them against the skeleton of that prefix leaves the Fineract and pooled medians unchanged.

Table 2. Skeleton cosine per project (C0, $N = 258$). The p-values are one-sided Wilcoxon tests against 0.85 in each direction.

Project	N	Median (IQR)	≥ 0.85 [95% CI]	$p_{>0.85}$	$p_{<0.85}$
Sylius	99	0.887 (0.076)	77.8% [68.3, 85.5]	< 0.001	1.000
Fineract	99	0.826 (0.062)	34.3% [25.1, 44.6]	1.000	< 0.001
Diaspora	60	0.831 (0.087)	41.7% [29.1, 55.1]	0.992	0.008
All	258	0.853 (0.085)	52.7% [46.4, 58.9]	0.474	0.526

4.3 Sensitivity to the Reference Point

Because 0.85 was not calibrated, we repeated the RQ1 test for reference points between 0.80 and 0.90 (Table S3 in the supplementary material). The pooled median lies significantly above every reference point up to 0.84 (at 0.84, sign test $p = 0.002$, Wilcoxon $p = 0.0097$) and above none from 0.85 upwards. The conclusion of RQ1 therefore depends on where the reference point is placed within a few hundredths. The project ranking does not: at every reference point Sylius has the largest share of files above it, and Fineract the smallest from 0.83 upwards.

We could not calibrate the reference point against defect labels, because the defect audit (Section 4.4) sampled only files at or above 0.85; within that range, cosine did not differ significantly between defective and non-defective files (Mann–Whitney $p = 0.21$).

4.4 Defects in High-Similarity Outputs

Defect audit. The audit is an exploratory observation and is not used to validate any conclusion, because its labels did not reach the agreement our protocol required (below). Of the 60 audited files, all with skeleton cosine of at least 0.85, 25 (41.7%, 95% CI [29.1, 55.1]) contain at least one defect of severity 2 or 3: 6 of 20 Sylius files, 16 of 20 Fineract files and 3 of 20 Diaspora files. The two facets flag exactly the same 25 files, so we report them as one result, not as two independent findings. The most frequent defects are wrong expected outcomes (20 files) and outcomes without a verifying Then (19). The share of 41.7% describes a sample balanced across projects and restricted to high-scoring files; it is not an estimate of the defect rate of the whole corpus. Section S3 of the supplementary material gives an example.

Agreement. On the 15 doubly labelled files, the independent rater agreed with the reviewed labels on 12 files for each facet. Cohen's κ was 0.571 (95% CI 0.05–1.00) for the business-logic facet and 0.609 (0.24–1.00) for the temporal/causal facet. Both values are below the threshold of 0.7 set in our protocol. Five of the six disagreements concern three Diaspora files that the independent rater marked as defective; in the consensus round the two raters adopted the independent rater's label in all six. After consensus, 6 of the 15 files have a business-logic defect and 8 a temporal/causal defect, against 5 and 5 in the reviewed labels. The other 45 files were not relabelled, so all audit figures above use the reviewed labels; with the consensus labels for the 15 files, 28 of the 60 files would be defective (26 on the business-logic facet).

Structural checks. The keyword-order check flags 5 of the 255 parseable generated files and 40 of the 244 parseable reference files. Generated files thus follow the Given–When–Then order more consistently than the references.

Perturbation test. In the 623 valid scenario pairs, no seeded defect lowered skeleton cosine below 0.8995, and changed numbers and quoted values leave the skeleton identical by construction. This shows insensitivity at the scenario level and does not estimate a false-negative rate for whole files.

4.5 Lexical Leakage from Reverse Engineering

The user stories are short compared with the references: a median of 4.9% of the distinct word trigrams of a reference file also occur in its story (Sylius 8.3%, Fineract 1.6%, Diaspora 5.8%). Higher overlap goes with higher similarity in two of the three projects: Spearman's ρ between overlap and skeleton cosine is 0.29 for Sylius (95% CI 0.11–0.46, $p = 0.003$), 0.33 for Diaspora (0.08–0.55, $p = 0.009$) and 0.11 for Fineract (-0.11 to 0.31, $p = 0.30$). The pooled value ($\rho = 0.43$, 0.33–0.53) is inflated by the project effect, since Sylius is high on both measures.

4.6 RQ2: Parser Compliance

RQ2 asks whether more than 80% of the generated files are valid Gherkin. Of the 258 non-exemplar files, 255 pass the parser (98.84%, 95% CI [96.64%, 99.76%],

$p < 0.001$; Table S4 in the supplementary material). The risk difference from the 80% reference rate is +18.84 percentage points (95% CI [16.64, 19.76], the Clopper–Pearson interval shifted by 0.80). Decision: reject H_0.

4.7 Prompt Ablation

The comparisons below use the same-time rerun (Table 3; 255 common files). Removing the example (C1) lowered skeleton cosine in 193 of the 255 files (median paired difference −0.030, 95% bootstrap CI −0.038 to −0.021; paired Wilcoxon, Holm-adjusted $p < 0.001$). The drop was largest in Fineract, where only 4 of 98 files reached 0.85. The difference is larger than the within-file SD across runs (0.020 for C0, 0.022 for C1; Section 4.8).

Table 3. Prompt ablation on the 255 files that are not an exemplar in any condition, in the same-time rerun (28 October 2026). Skeleton cosine medians overall and per project (n = 98, 98, 59), share of files ≥ 0.85 with 95% CI, and parser PASS count. Table S7 in the supplementary material gives the original runs.

Condition	Median	≥ 0.85 [95% CI]	Sylius	Fineract	Diaspora	PASS
C0 (exemplars 2, 102, 202)	0.852	52.5% [46.2, 58.8]	0.892	0.829	0.829	255
C1 zero-shot	0.811	33.7% [27.9, 39.9]	0.872	0.779	0.830	252
C2 (exemplars 86, 119, 203)	0.873	63.5% [57.3, 69.4]	0.891	0.873	0.841	254

Replacing the exemplars (C2) raised cosine in 140 files and lowered it in 115 (median paired difference +0.008, 95% CI −0.001 to 0.015; Holm-adjusted $p < 0.001$; the Wilcoxon test concerns the pseudo-median of the differences, the interval their median). The C2 median of 0.873 comes mostly from Fineract (median 0.829 under C0, 0.873 under C2), while Sylius and Diaspora change by less than 0.015. The paired difference is smaller than the within-file SD across runs (0.020 for C0, 0.019 for C2), so by the rule fixed in the plan we do not report it as an effect of the exemplars.

Parser compliance was similar under all three prompts. Under C0, C1 and C2, 255, 252 and 254 of the 255 files parse (exact McNemar against C0, Holm-adjusted $p = 0.50$ and 1.0); the tests do not establish equivalence. The original ablation, with C0 run three months before C1 and C2, gave the same direction and similar paired differences (−0.028 and +0.008; Table S7 in the supplementary material).

4.8 Run-to-Run Variation

Repeating a prompt rarely reproduced its output, but the scores are stable: on the 60 repeated files ICC(1,1) is 0.921, 0.904 and 0.929 for C0, C1 and C2, and the pooled within-file SD is 0.020, 0.022 and 0.019. The parser outcome changed between runs for 0, 2 and 2 of the 60 files, but 19 (32%), 13 (22%) and 14 (23%) files lay on both sides of 0.85 in different runs. Between the new and the original C0, 45 files (17%) lie on different sides of 0.85, yet the decisions do not change: the new C0 has median 0.852 (sign test $p = 0.247$), and 258 of 258 files parse.

The mean between-prompt effect (C2 minus C0) is +0.004 (95% CI −0.011 to 0.020), and the per-file Welch 95% interval of the difference lies wholly above 0 for 19 files, wholly below 0 for 17 and includes 0 for 24 (unadjusted). Changing the exemplars thus moves single files by more than repeating the prompt does, but in both directions, with no detectable average effect.

5 Discussion

5.1 Why Parser Compliance and Structural Similarity Diverge

Nearly all generated files are valid Gherkin (255 of 258), consistent with prior work showing that examples reduce syntax errors [2] and with industrial studies reporting high syntactic correctness of generated Gherkin [5] and test code [3]; the prompt ablation shows that this does not require the example (Section 4.7). Matching the reference structure is harder, and the per-project results locate the difficulty: Fineract files are long and data-driven, and 94 of its 99 analysed references are cut by the encoder's 256-token limit, while Diaspora files depend on browser-level steps that a story does not state; the truncated story input of 84 Fineract files does not explain the gap (Section 4.2). The experiment does not isolate these factors, but a pooled median that averages one project above the reference point with two below it is not a meaningful summary of capability. The similarity result is also more cautious than studies reporting strong semantic or text-similarity results [1,4], partly because this experiment uses a different metric, a fixed reference point, a multi-domain sample and a hypothesis test; cross-paper metric values are contextual comparisons, not model rankings.

5.2 What the Similarity Metric Can and Cannot Show

The ablation and the repeated runs show how fragile the RQ1 decision is: repeating the same prompt moves a fifth to a third of the 60 repeated files across 0.85. A different exemplar raises the median from 0.852 to 0.873, mostly in Fineract, but its paired effect stays within this run-to-run variation, while removing the example lowers the score by more than it. We conjecture that this follows from truncation: the encoder sees only the first 256 tokens of 94 of the 99 Fineract references, so an exemplar with a typical opening can raise the score without evidence that the business content improved; the ablation does not test this.

Skeleton cosine is a measure of structural resemblance, not of correct behaviour. In the exploratory audit, 25 of 60 high-similarity files contain a defect that changes what the test checks, cosine did not rank defective files significantly differently, seeded defects leave the score at or above 0.8995, and changed numbers or values cannot change it at all. A threshold on this metric, whether 0.85 or any value between 0.80 and 0.90, therefore cannot certify that a generated file preserves its acceptance criteria, and high embedding similarity alone, in any of the studies of Table S5, says little about it. We report 0.85 as a descriptive reference point and do not interpret files above it as correct. The overlap correlations in two projects

(Section 4.5) suggest that part of the similarity may also come from wording that the reverse-engineered story carries over from its reference; they do not establish a causal effect.

5.3 Practical Use in BDD Workflows

For stories of this kind, the evidence supports a bounded workflow: use GPT-5.4-mini to draft valid Gherkin, then have a reviewer check expected outcomes, conditions, data values and the coverage of each acceptance criterion. The audit suggests where review effort is needed most: wrong expected outcomes and missing verifying Then steps were the most frequent defects, and defective files were far more frequent in the data-heavy finance domain (16 of 20) than in the other two (6 and 3 of 20). The experiment does not justify promoting generated scenarios into CI/CD as authoritative acceptance tests.

6 Threats to Validity

For each threat we state the mitigation we actually performed and the risk that remains. Threats that we could only measure or disclose are collected at the end of the section.

6.1 Internal Validity

Reverse-engineered stories. The stories were written by the model under test from the reference files, so the input can carry wording, structure and scenario order from the file it is compared with. *Mitigation:* we measured story–reference overlap and its correlation with similarity (Section 4.5) and checked the Fineract stories written from a prefix (Section 4.2). *Residual risk:* overlap correlates with similarity in two of the three projects, structural leakage is not measured, and the results may overstate performance on natural requirements.

Exemplars, pilot and ablation. *Mitigation:* the three original exemplars are excluded from the primary analysis, both denominators are reported, and the same-time rerun repeats C0, C1 and C2 with interleaved requests. *Residual risk:* the 30 pilot stories used when the reference point was set are part of the evaluated set, and C2 uses one random draw of exemplars.

Hosted model and output cap. *Mitigation:* we pinned the snapshot, used temperature 0, logged each call, measured run-to-run variation (Section 4.8) and ran the proposal's cap of 1,500 output tokens as C0cap. *Residual risk:* nondeterminism remains, changes of the serving configuration cannot be detected, and the RQ2 result applies to uncapped generation; under the cap, long files are cut and parser compliance is lower.

6.2 Construct Validity

Similarity metric. Skeleton cosine is an imperfect proxy for business correctness. *Mitigation:* we checked it against defect labels, seeded defects, reference points

from 0.80 to 0.90 and per project. *Residual risk:* it is blind to numbers and quoted values, ignores data tables and Examples, often sees only the start of a file, and did not separate defective from non-defective audited files.

Defect labels. *Mitigation:* an independent human rater labelled 15 files, we report κ with 95% intervals, and the two raters held the consensus round our protocol requires on these files (Section 4.4). *Residual risk:* the labels are AI-drafted and were reviewed in one confirmation; $\kappa = 0.571$ and 0.609 are below the protocol's 0.7 and imprecise with 15 files; the other 45 files were not relabelled, so the audit is exploratory and may undercount defects, in particular in the social-network domain; and it covers only files at or above 0.85.

Perturbation test and parser compliance. *Mitigation:* non-parsing pairs are excluded, results are reported by operator, and validity is checked with behave's parser rather than a dry run. *Residual risk:* the perturbation pairs are single, dependent scenarios and do not estimate whole-file sensitivity, and a parser pass shows only valid Gherkin, not execution against the application.

6.3 External and Conclusion Validity

Projects. *Mitigation:* all main results are reported per project. *Residual risk:* three mature, English-language public projects are not representative of BDD practice, and because the projects contribute unequal numbers of files and differ significantly in similarity, pooled results do not generalise across domains.

Thresholds and tests. *Mitigation:* we report how the conclusions change between reference points of 0.80 and 0.90, and the ablation and rerun plans were fixed with their hashes before the runs. *Residual risk:* neither the 0.85 reference point nor the 80% threshold is an industry standard, the reference point was not calibrated, and the RQ1 test result changes between 0.84 and 0.85. All analyses planned after the reviews are exploratory; their p-values are not corrected.

Uncontrolled limitations. The measures above do not control every threat. We had no independently written requirements, could not detect changes in the hosted service, did not execute the generated files against step definitions, and could not relabel the 45 audit files outside the consensus round. The conclusions therefore hold only for reverse-engineered stories from these three projects, in the time window of the runs, and with the similarity metric read as a proxy.

7 Conclusion

This study evaluated how well GPT-5.4-mini reconstructs 261 feature files of three open-source projects from Connextra-style user stories reverse-engineered from them ($N = 258$ without the three prompt exemplars). The main contribution is an auditable multi-domain evaluation that separates parser compliance from structural resemblance and demonstrates, through defect auditing and perturbation, why embedding similarity cannot certify business correctness. For RQ1, the median skeleton cosine similarity was 0.853 and the one-sided exact sign test gave $p = 0.209$ ($\delta = 0.054$; Wilcoxon $p = 0.474$), so we fail to reject H_0 and do not

claim that GPT-5.4-mini exceeds the reference point of 0.85; only the e-commerce project exceeded it, and the conclusion would change at 0.84. For RQ2, 255 of 258 generated files were valid Gherkin (98.84%, $p < 0.001$), so we reject H_0. In an exploratory audit, 25 of 60 files at or above the reference point contained a business-logic defect, and seeded defects left the score almost unchanged. These labels are AI-drafted and human-reviewed, their agreement with an independent rater was below our protocol's threshold of 0.7, and a consensus round raised the defective files among the 15 doubly labelled ones from 5 to 8. In a same-time rerun, removing the example lowered the median from 0.852 to 0.811, while a different exemplar had a paired effect within the run-to-run variation.

For practice, and pending a test on independently written stories, the supported use case is assisted drafting: a QA engineer can start from a valid feature file and focus review on expected outcomes, conditions, data values and criterion coverage. The defect evidence behind this advice is exploratory, comes from a high-similarity sample stratified by project, and does not estimate the defect rate of the whole corpus. Neither a parser pass nor a high similarity score supports autonomous acceptance. Future work should (1) use independently written requirements, (2) validate semantics with criterion-level review by independent raters or with execution against step definitions, and (3) compare retrieval-augmented and open-source baselines on the same files.

Acknowledgments. We thank Tran Tuan Khang, Vo Hieu Chuong and Tong Vy Thuan for their contributions to the first version of this study. GPT-5.4-mini is the system under evaluation and was also used to write the user stories from the reference files (Section 3). In the revision, an AI coding assistant (OpenAI Codex) drafted the 60 defect-audit labels, which an author reviewed, and AI assistants (OpenAI Codex and Anthropic Claude) helped write analysis scripts and draft text; Claude also prepared reading aids for the 15-file rater (rubric notes, an acceptance-criteria map, and reading sheets and a glossary for Fineract) that contained no labels. NotebookLM was used to locate passages in the related-work sources; every quoted fact was checked against the source PDFs. All numbers were recomputed from the released data and scripts, and the authors take responsibility for the content.

Data and Ethics. This study involves no human participants; the empirical subjects are public software artifacts and model-generated Gherkin scenarios. Human-subjects review is therefore not applicable. The replication package contains the dataset, prompts, scripts, generated outputs, API logs, audit labels and additional analyses at https://github.com/simonhoang611/RT-SWT-004-Group4. The supplementary material is at https://github.com/simonhoang611/RT-SWT-004-Group4/blob/main/paper_revised/367005_Evaluating_GPT_5_4_mini_for_Gherkin_Test_Generation_Supplementary_Material.pdf.

Disclosure of Interests. The authors declare no external funding and no conflicts of interest.

References

1. Rathnayake, A., Shahin, M., Abaei, G.: Behaviour Driven Development Scenario Generation with Large Language Models. arXiv:2603.04729 (2026). https://doi.org/10.48550/arXiv.2603.04729
2. Karpurapu, S., Myneni, S., Nettur, U., Gajja, L.S., Burke, D., Stiehm, T., Payne, J.: Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. IEEE Access 12, 58715–58721 (2024). https://doi.org/10.1109/ACCESS.2024.3391815
3. Ferreira, M., Viegas, L., Faria, J.P., Lima, B.: Acceptance Test Generation with Large Language Models: An Industrial Case Study. In: IEEE/ACM International Conference on Automation of Software Test (AST), pp. 1–11 (2025). https://doi.org/10.1109/AST66626.2025.00007
4. Fernandes, H., Perkusich, M., Albuquerque, D., Silva, I., Santos, D.F.S., Gorgônio, K., Perkusich, A.: A Comparative Study of LLMs for Gherkin Generation. In: Brazilian Symposium on Software Engineering (SBES), pp. 171–181 (2025). https://doi.org/10.5753/SBES.2025.9888
5. Fonseca, P.L., Lima, B., Faria, J.P.: Streamlining Acceptance Test Generation for Mobile Applications Through Large Language Models: An Industrial Case Study. In: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 3311–3321 (2025). https://doi.org/10.1109/ASE63991.2025.00273
6. Zhang, Q., Shang, Y., Fang, C., Gu, S., Yu, S., Zhou, J., Chen, Z.: TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. Frontiers of Computer Science 21(6), 2106202 (2027), published online 13 October 2025. https://journal.hep.com.cn/fcs/EN/10.1007/s11704-025-50078-9, last accessed 2026/10/28.
7. Chang, H.-F., Shokrolah Shirazi, M.: A Systematic Approach for Assessing Large Language Models' Test Case Generation Capability. Software 4(1), 5 (2025). https://doi.org/10.3390/software4010005
8. Wang, Z., Wang, W., Li, Z., Wang, L., Yi, C., Xu, X., Cao, L., Su, H., Chen, S., Zhou, J.: XUAT-Copilot: Multi-Agent Collaborative System for Automated User Acceptance Testing with Large Language Model. arXiv:2401.02705 (2024). https://doi.org/10.48550/arXiv.2401.02705
9. Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023), pp. 21558–21572 (2023). https://doi.org/10.52202/075280-0943
10. Tong, W., Zhang, T.: CodeJudge: Evaluating Code Generation with Large Language Models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 20032–20051 (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1118
11. Kang, S., Yoon, J., Yoo, S.: Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. In: 45th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 2312–2323 (2023). https://doi.org/10.1109/ICSE48619.2023.00194
12. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Lahiri, S.K.: LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. IEEE Transactions on Software Engineering 50(9), 2254–2268 (2024). https://doi.org/10.1109/TSE.2024.3428972

13. Harman, M., Ritchey, J., Harper, I., Sengupta, S., Mao, K., Gulati, A., Foster, C., Robert, H.: Mutation-Guided LLM-based Test Generation at Meta. In: Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion), pp. 180–191 (2025). https://doi.org/10.1145/3696630.3728544
14. Zadenoori, M.A., Dąbrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv:2509.11446 (2025). https://doi.org/10.48550/arXiv.2509.11446
15. Baltes, S., Angermeir, F., Arora, C., Muñoz Barón, M., Chen, C., Böhme, L., Calefato, F., Ernst, N., Falessi, D., Fitzgerald, B., Fucci, D., He, J., Treude, C., Kalinowski, M., Lambiase, S., Russo, D., Lungu, M., Martinez Montes, C., Prechelt, L., Ralph, P., van Tonder, R., Wagner, S.: Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv:2508.15503v8 (2026). https://doi.org/10.48550/arXiv.2508.15503
16. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: EMNLP-IJCNLP, pp. 3982–3992 (2019). https://doi.org/10.18653/v1/D19-1410
17. Sylius: Sylius, branch 2.3, commit 9ab28ca. https://github.com/Sylius/Sylius/tree/9ab28ca4b5ad5797216fe14e72e4d6cfe29c3fdb, last accessed 2026/10/28.
18. Apache Software Foundation: Apache Fineract, branch develop, commit 5a93304. https://github.com/apache/fineract/tree/5a93304677061501f5d0c243b8dbf84752e08b0b, last accessed 2026/10/28.
19. diaspora* contributors: diaspora*, branch develop, commit b7bf1dc. https://github.com/diaspora/diaspora/tree/b7bf1dc33566c036a3d788dd8876e97c30905842, last accessed 2026/10/28.
20. OpenAI: GPT-5.4 Mini model documentation. https://developers.openai.com/api/docs/models/gpt-5.4-mini, last accessed 2026/10/28.
21. OpenAI: Create chat completion, API reference. https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create, last accessed 2026/10/28.
22. Sentence Transformers: all-MiniLM-L6-v2 model card, revision 1110a24. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/tree/1110a243fdf4706b3f48f1d95db1a4f5529b4d41, last accessed 2026/10/28.
23. behave developers: behave v1.3.3 release. https://github.com/behave/behave/releases/tag/v1.3.3, last accessed 2026/10/28.
24. Cohen, J.: A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement 20(1), 37–46 (1960). https://doi.org/10.1177/001316446002000104
25. Arcuri, A., Briand, L.C.: A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In: 33rd International Conference on Software Engineering, pp. 1–10 (2011). https://doi.org/10.1145/1985793.1985795
26. Agresti, A.: Categorical Data Analysis, 2nd edn. John Wiley & Sons (2002). https://doi.org/10.1002/0471249688
27. Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Monographs on Statistics and Applied Probability, vol. 57. Chapman & Hall, New York (1993)

28. Clopper, C.J., Pearson, E.S.: The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. Biometrika 26(4), 404–413 (1934). https://doi.org/10.1093/biomet/26.4.404
29. Kruskal, W.H., Wallis, W.A.: Use of Ranks in One-Criterion Variance Analysis. Journal of the American Statistical Association 47(260), 583–621 (1952). https://doi.org/10.1080/01621459.1952.10483441
30. Holm, S.: A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics 6(2), 65–70 (1979). https://www.jstor.org/stable/4615733